

准考证号

题号	一	二	三	四	五	六	总分
得分							
阅卷人							

注：本试卷满分 100 分，折合为 40 分计入总成绩。

随着雾霾对日常生活影响的日益加深，互联网用户越来越需要一款能够实时查询雾霾指数的信息系统。由于雾霾中的主要成分为可吸入颗粒物，因此本查询系统仅以不同的可吸入颗粒物的浓度值作为查询指标，查询的指标分别为 PM2.5、PM10、PM100(PM2.5、PM10、PM100 分别代表颗粒物的直径)。

此查询系统的需求描述如下：

1、该查询系统采用 B/S 结构，用户可从浏览器使用该查询系统的功能，也可通过手机 APP 的形式访问。

2、该查询系统能够定时从位于城市不同区域的监测站系统中读取三种指标的采集浓度值，并将浓度值存储到服务器中。

3、当用户作为游客使用系统时，有以下功能：

①查询指标值：系统提供城市列表，用户可从城市列表中选择城市，以浏览该城市当前的三种指标值。针对每种指标值，系统可展示的信息包括：指标的描述信息（包括：指标名称、指标形成原因、指标的危害）、三种指标的最近采集数据（包括：采集区域名称、指标在该区域的采集浓度、采集时间、采集日期）、城市指标综合浓度值（某指标在一个城市不同区域的采集浓度平均值）、当前综合浓度下的危害等级、在该危害等级下的出行建议。

4、用户可在系统中注册，注册时需提供登陆名、密码、电话号码、常驻城市信息。

5、当用户登陆系统后，可使用以下特殊功能：

①手机绑定：可以将用户在注册时填写的手机号与本查询系统进行绑定。用户可自行定义信息推送的间隔时间，从每天零时开始，根据用户定义的间隔时间，系统以短信形式向用户推送用户指定所在地或由用户手机定位系统得到的所在地的空气指标的危害等级以及相应出行建议。

②用户评价：用户在使用查询指标值功能时，可以对查询得到的“城市指标综合浓度值”持肯定“点赞”或否定“踩”意见，并且当用户选择“踩”时，用户还可根据自身体验给出一个体验值（即用户体验下某指标的近似浓度），并将该体验值对其它登陆用户公开。

6、系统设管理员，管理员具有以下功能：

①可以设置数据采集的时长（如每隔 30 分钟从监测站读取数据）、可以修改城市列表、可以修改指标的描述信息。由于系统中可能存在多个管理员，应记录修改来源于哪位管理员。

②设置危害等级：为直观地描述各指标的危害程度，管理员可为每个指标设置危害等级，目前设置了三种危害等级：绿色、黄色、红色，并可对三种指标的危害等级进行配置。

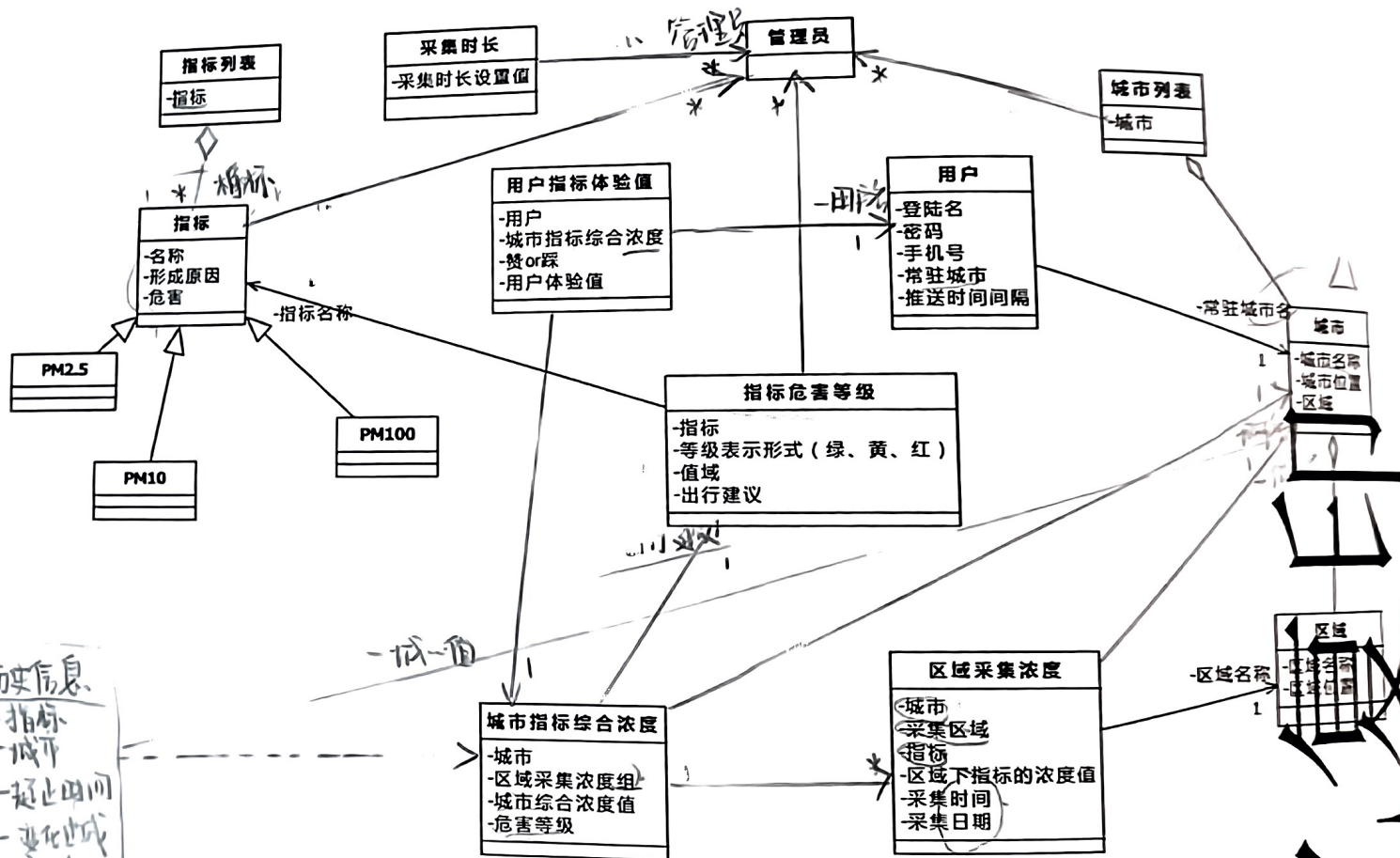
一 (15分)需求获取与敏捷开发

在第一列列出该软件的全体参与者，分析各自的用例清单，并评估各用例的优先级(5、4、3、2、1)。用例名称请采用简洁明了的中文短语，后附用例的优先级，下表中已给出一个样例，仿照该样例填写。注：下表各行未必全部填满。

参与者名称	用例（优先级）
普通用户 游客	注册 (5)，查询指标值 (4)
登陆用户	绑定手机 (3) 定义信息推送的间隔时间 (2) 查询指标值 (4) 评价数据 (1) 给出体验值 (1)
管理员	设置数据采集时长 (4) 修改城市列表 (4) 设置危害等级 (4) 修改指标描述信息 (2)
监测站系统	读取检测值 (5)
短信系统	发送推送信息(或推送危害等级出行建议) (3)
手机定位系统	获取位置 (3)

纸张记忆复印

△ 实例化(类) ☆ 是整个类名



(2) (5分) 用户希望此查询系统提供统计信息，因此希望系统实现以下扩充功能：

查询历史信息并统计：用户可对一段时间内的指标值进行统计，用户需指定某一指标和城市，然后输入起止时间，系统即展示输入时间段内该指标的“城市指标综合浓度”随采集时间的变化曲线，并统计在该输入时间段内综合浓度的最大值、最小值和平均值，之后系统将指标曲线和统计结果存入到数据库中。

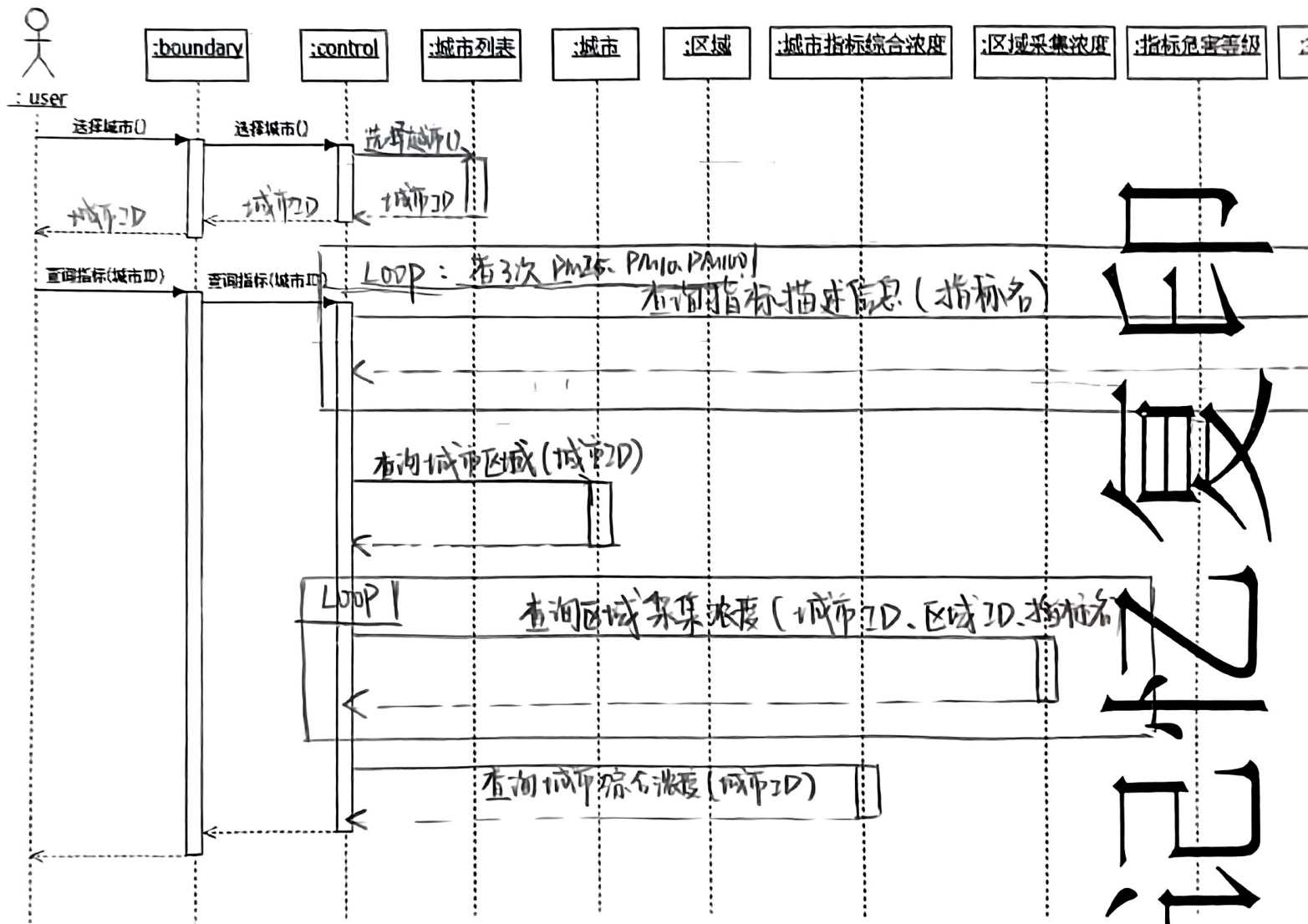
为了实现该功能，在已有实体类的前提下，需要引入哪个(哪些)新的实体类？给出新实体类的名称、属性集合及各属性的可见性，以及它(们)与其他已有各实体类之间的关系(继承、关联、组合、聚合)。结果直接在上面的实体类图中画出。

△ (3分?)

三(13分)时序图

请实现系统描述中的“第3个需求描述中①查询指标值”对应的时序图 (Sequence Diagram), 下图中已给出了时序图中所需要的所有对象和部分消息, 请补充为完成该功能所需要的剩余消息 (消息要包含消息描述和参数)。

注: ①无需补充对象; ②注意区分发送消息和返回消息。



纸张记忆复印

非功能性需求

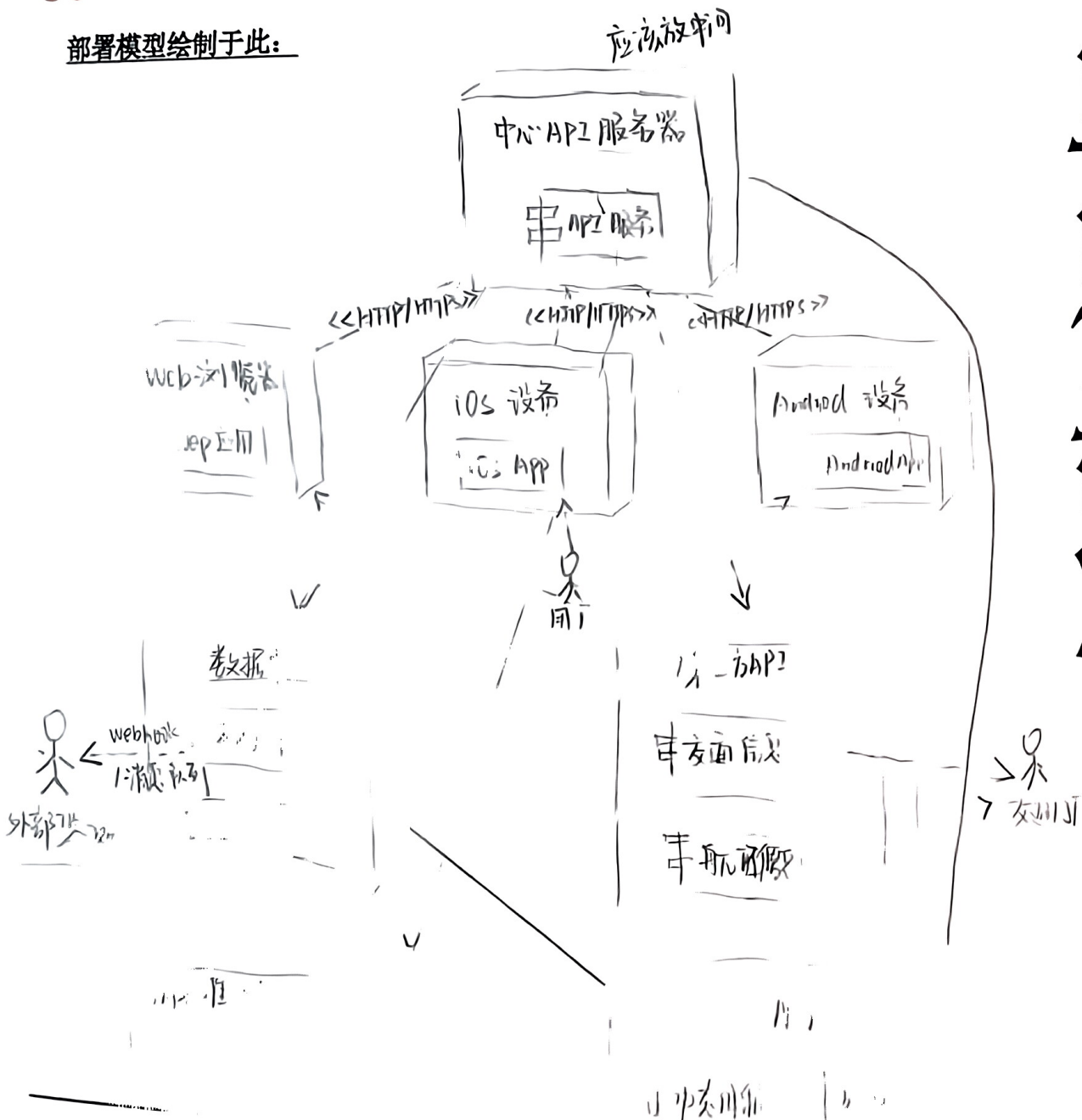
四 (15分) 面向 NFR 的架构设计

“雾霾指数查询系统”在架构设计方面有如下需求：

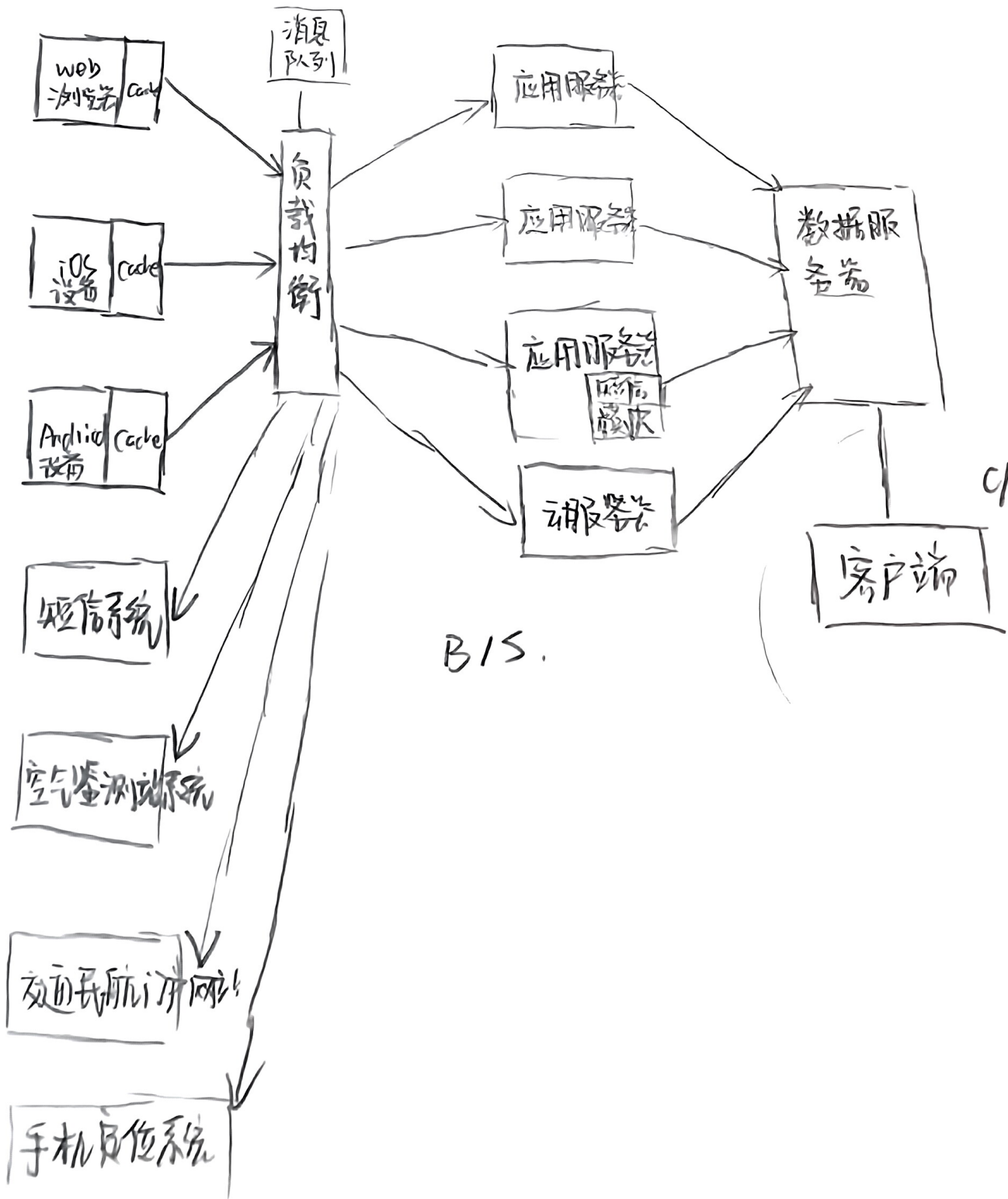
1. 系统除了能够支持 Web 端访问外，还提供了 iOS 和 Andriod 平台上的客户端 APP，此类 APP 可借助手机定位系统对用户进行定位，方便用户查询和信息推送。不同的客户端界面展示方式不同，但是业务逻辑基本一样，要尽量避免重复开发。
2. 系统能够通过接口从各城市各区域的空气监测站系统中获取数据，各监测站采集数据的时间点和频度不尽相同，接口设计时要保证系统获取数据的效率和实时性（例：如果系统接口频繁扫描监测站是可以保证得到的数据是最新的，但是显然会增大系统的负担，降低系统的效率）。
3. 系统能够从交通、民航等部门的门户网站上获取同雾霾相关的信息，如单双号限行、航班推迟等信息，供用户查询。
4. 系统能够通过短信形式，推送消息给订阅用户。
5. 在空气质量较差的时期（如供暖期），访问量会大幅度增加，而空气质量较好的时期，访问量则会较小。要保证服务器资源的合理利用，既能保证大量访问时的性能，又能做到空闲时节省资源。

针对上述情况，使用 UML 部署模型描述系统的物理架构，并简要阐述上述各项需求是如何被解决的，采用了何种策略，相应接口应如何设计。

部署模型绘制于此：



印 复 忆 记 张 纸



五 (20 分) 软件测试

1. (8 分) 系统可从设置在各区域的空气监测站系统中读取各时间点采集到的雾霾指标值, 读取的数据格式遵循以下规则:

- (a) 分为三部分, 依次为: 区域编号、采集时间点、指标值, 三部分之间采用 "@" 符号进行相连;
 (b) "区域编号" 部分的长度为 3, 由字母或者数字构成;
 (c) "采集时间点" 部分, 由长度为 2 的数字构成, 取值范围为 00~23;
 (d) "指标值" 部分, 均为数字, 取值范围为 0~1000。

对符合上述规则的数据, 模块返回结果为 "合法", 否则返回 "非法"。

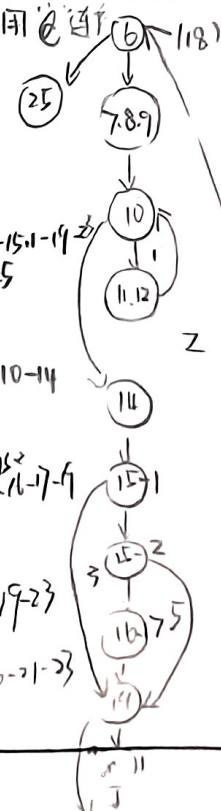
利用黑盒测试的相关知识, 完成以下题目:

- (1) (4 分) 使用等价类划分方法, 以表格形式给出所有的有效等价类和无效等价类;
 (2) (4 分) 设计一组黑盒测试用例 (包括输入数据、覆盖的等价类、期望的输出结果)。

输入条件	有效等价类	无效等价类
"区域编号" 长度	3 (11)	≠3 (18)
"区域编号" 组成	字母与数字 (12)	含有非字母与数字 (19)
采集时间点 长度	2 (13)	≠2 (10)
组成	数字 (14)	含非数字 (11)
取值范围	00~23 (15)	<00 (12) >23 (13)
"指标值" 组成	数字 (16)	含非数字 (14)
取值范围	0~1000	<0 (15) >1000 (16)
指标的组成	有三部分: 区域编号、采集时间点、指标值, 且三部分用 "@" 相连 (17)	没有三部分组成 (17) 没有采用 "@" 相连 (18)

用例号	输入数据	等价类
123 @ 00 @ 10		(11) (12) (14) (15) (16) (17)
12 @ 00 @ 10		(18)
1-2 @ 00 @ 10		(19)
123 @ 0 @ 10		(10)
123 @ ab @ 10		(11)
123 @ -12 @ 10		(12)
123 @ 25 @ 10		(13)
123 @ 0 @ 0		(14)
123 @ 0 @ 0		(15)
123 @ 0 @ 0		(16)

- ① 6-25
 ② 6-7-8-9-10-14-15-1-19-6-25
 ③ 6-7-8-9-10-11-12-10-14-16-19-23-6-25
 ④ 6-7-8-9-10-14-15-16-17-1-23-6-25
 ⑤ 15-2-19-23
 ⑥ 1-7-20-21-23



纸张记忆复印

2. (12)“雾霾指标查询”系统中有一个函数 `getBestWorstArea()`，其根据各区域采集来的 T 个时间点的 $PM_{2.5}$ 指标值，计算得出在时间段 T 内，达到绿色等级（平均指标值 < 100 ）的区域中空气质量最好的区域，以及达到红色等级（平均指标值 > 500 ）的区域中空气质量最差的区域。其输入为一个字符串数组 `pmValues` 和时段 T ，`pmValues` 中的元素为不同区域不同时间点的采集数据，数据格式为“区域编号@采集时间点@ $PM_{2.5}$ 指标值”， T 为整数（代表每个区域均采集 T 个时间点的数据），输出为空气质量最好和最差区域的编号。

程序伪代码如下：

```
String getBestWorstArea(String[] pmValues, int T) {
    1  pmValues.sort();    //按照升序对 pmValues 中字符串进行排序
    2  String worstArea = "noRed", bestArea = "noGreen", curArea = "";
    3  float maxAvg = 0, minAvg=100, newSumValue = 0, avgValue = 0;
    4  String[] pmValue=null;
    5  int i= 0;

    6  while(i<pmValues.length){
    7      pmValue = pmValues[i].split("@");    //split 函数按照参数分割字符串为子串数组
    8      curArea = pmValue[0];
    9      newSumValue = 0;

    10     for (int j=0;j<T;j++){
    11         pmValue = pmValues[i+j].split("@");
    12         newSumValue += Float.valueOf(pmValue[2]); //Float.valueOf() 将字符串转为浮点数
    13     }

    14     avgValue = newSumValue / T;
    15     if ((avgValue > maxAvg) && (avgValue > 500)){
    16         worstArea = curArea;
    17         maxAvg = avgValue;
    18     }
    19     if (avgValue < minAvg){
    20         bestArea = curArea;
    21         minAvg = avgValue;
    22     }
    23     i+=T;
    24 }

    25     return "最好的绿色等级区域是:" + bestArea + " && 最差的红色等级区域是:" + worstArea;
    26 }
```

基于上述源代码，利用白盒测试方法，完成以下题目：

(1) (3 分) 绘制该程序的控制流程图；

(2) (2 分) 计算其环形复杂度（圈复杂度）；

(3) (4 分) 列出所有的基本路径；

(4) (3 分) 针对每条基本路径来设计测试用例，测试用例中至少应包含输入数据和期望结果。

输入

输出

最好的绿色等级区域是 123 22 最差的红色等级区域是

1-189-10-1
-151-19-